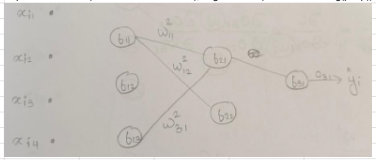
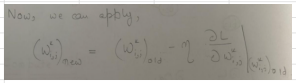
Loss Functions	Squared error loss (normal error distribution), Logistic Loss (link function of log(p/1-p))	
Overall Network		
Updates using SGD		
	Forward pass: Using the dat to pass through the network to get loss function value Backward pass: 1. Compute PD (partial gradients), 2. Updating weights using SGD.	
Updation step	We start with random weights, do forward pass then do backward pass -----> which leads to updated weights and then repeat the process again and again	
Most Used activation function	Sigmoid: (1/1+exp(-x))	Derivative of sigmoid: f(x) * (1-f(x))
	Tanh: exp(x) - exp(-x) / exp(x) + exp(-x)	Derivative of tanh: 1-f(x)^2
Vanishing and Exploding Gradients	**Setup** • You have **3 layers**, each with 1 neuron • You use **sigmoid activation**: $$\sigma(z) = \frac{1}{1+e^{-z}}, \quad \sigma'(z) = \sigma(z)(1-\sigma(z))$$ • The input is x, and we compute the output a_3 as: $$a_3 = \sigma(w_3 \cdot \sigma(w_2 \cdot \sigma(w_1 \cdot x)))$$ **What Each Term Means** Term / Meaning x — Input value w_1, w_2, w_3 — Weights for each layer $z_1 = w_1 \cdot x$ — Pre-activation of layer 1 $a_1 = \sigma(z_1)$ — Activation (output of layer 1 $z_2 = w_2 \cdot a_1$ — Pre-activation of layer 2 $a_2 = \sigma(z_2)$ — Activation of layer 2 $z_3 = w_3 \cdot a_2$ — Pre-activation of output $a_3 = \sigma(z_3)$ — Final prediction	**Backpropagation: Computing the Gradient** We want to find how the final loss depends on w_1, the weight in the **first layer**: $$\frac{dL}{dw_1} = \frac{dL}{da_3} \cdot \frac{da_3}{dz_3} \cdot \frac{dz_3}{da_2} \cdot \frac{da_2}{dz_2} \cdot \frac{dz_2}{da_1} \cdot \frac{da_1}{dz_1} \cdot \frac{dz_1}{dw_1}$$ All the $\frac{da}{dz}$ are **sigmoid derivatives**: $$\frac{da}{dz} = \sigma(z)(1-\sigma(z)) \in (0, 0.25)$$ **Why It's a Problem** If: • $\sigma'(z) = 0.25$ at most, • Then after multiplying 3 layers: $$\text{Gradient} \approx (0.25)^3 = 0.0156$$ If you go deeper (say 10 layers): $$\text{Gradient} \approx (0.25)^{10} = 0.00000095$$ So the gradient becomes tiny, meaning **weights in early layers barely change → they don't learn!**
	Forward Pass: Let the input be x, and each layer has one weight w_1, w_2, w_3: $$z_1 = w_1 \cdot x$$ $$z_2 = w_2 \cdot z_1 = w_2 \cdot w_1 \cdot x$$ $$z_3 = w_3 \cdot z_2 = w_3 \cdot w_2 \cdot w_1 \cdot x = \hat{y}$$ No activation → just multiplication through the layers.	**Loss Function: Mean Squared Error** $$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(z_3 - y)^2$$ **Backpropagation (Gradient of Loss w.r.t. w_1)** We'll compute $\frac{dL}{dw_1}$ using the chain rule: $$\frac{dL}{dw_1} = \frac{dL}{dz_3} \cdot \frac{dz_3}{dz_2} \cdot \frac{dz_2}{dz_1} \cdot \frac{dz_1}{dw_1}$$ **Each term:** • $\frac{dL}{dz_3} = z_3 - y$ • $\frac{dz_3}{dz_2} = w_3$ • $\frac{dz_2}{dz_1} = w_2$ • $\frac{dz_1}{dw_1} = x$ So: $$\frac{dL}{dw_1} = (z_3 - y) \cdot w_3 \cdot w_2 \cdot x$$
Dropouts No scaling is required at the time of inference (because all nodes will be active)	**Breakdown** **1. Forward Pass (During Training):** • A **random binary mask** is generated. • Neurons are **dropped (set to 0)** based on the dropout probability. • Their outputs are **not passed** to the next layer. • The remaining active neurons are **scaled up** by $\frac{1}{1-p}$ to preserve the expected value. So: $$a_{\text{dropped}} = \frac{a \cdot m}{1-p}$$ Where: • a: activations • m: binary mask (same shape as a) • p: dropout rate	**2. Backward Pass (Backpropagation):** • The mask **from the forward pass is reused**. • Gradients are **only computed for the active neurons** (those not dropped). • Dropped neurons get **no gradient** (because their output was 0 in the forward pass). So dropout **affects** the backward pass **indirectly**, by removing nodes from the forward computation graph.
CBOW, SKIP GRAM	**3. Word Embeddings (Word2Vec, GloVe, FastText)** **Goal:** Learn *dense, low-dimensional* vectors where semantic similarity is preserved. **3.1 Word2Vec** Two architectures: 1. **Skip-gram** — predict context words given a center word. $$\max \frac{1}{T} \sum_{t=1}^{T} \sum_{-c \le j \le c, j \ne 0} \log p(w_{t+j} \mid w_t)$$ 2. **CBOW** — predict the center word from surrounding words. $$\max \frac{1}{T} \sum_{t=1}^{T} \log p(w_t \mid w_{t-c}, \ldots, w_{t+c})$$ **Softmax probability:** $$p(w_O \mid w_I) = \frac{\exp(\mathbf{v}'_{w_O} \cdot \mathbf{v}_{w_I})}{\sum_{w=1}^{V} \exp(\mathbf{v}'_{w} \cdot \mathbf{v}_{w_I})}$$ (Usually replaced by **Negative Sampling** to speed training.)	**2. Problem with Full Softmax** The denominator sums over all V words in the vocabulary — expensive when V is millions. We want to avoid computing it. **3. Reframing as Binary Classification** Instead of computing a multi-class probability distribution, we reframe the problem: • **Positive sample:** (w_t, w_c) is a real co-occurrence → label $y = 1$ • **Negative samples:** (w_t, w_i) where w_i is randomly chosen → label $y = 0$ This becomes **k+1 binary classification problems** per center word. **4. Logistic Regression for a Pair** For any center word w_t and candidate context w, we model $P(D = 1 \mid w_t, w)$ — the probability that w is a true context — as: $$P(D = 1 \mid w_t, w) = \sigma(\mathbf{v}'_w \cdot \mathbf{v}_{w_t})$$ $$P(D = 0 \mid w_t, w) = 1 - \sigma(\mathbf{v}'_w \cdot \mathbf{v}_{w_t}) = \sigma(-\mathbf{v}'_w \cdot \mathbf{v}_{w_t})$$ where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid. **5. Likelihood for One Positive + k Negatives** If we have: • One positive pair: (w_t, w_c) • k negative pairs: (w_t, w_i) where $w_i \sim P_n(w)$ The likelihood is: $$L_{\text{likelihood}} = P(D = 1 \mid w_t, w_c) \cdot \prod_{i=1}^{k} P(D = 0 \mid w_t, w_i)$$ Substitute the sigmoid forms: $$L_{\text{likelihood}} = \sigma(\mathbf{v}'_{w_c} \cdot \mathbf{v}_{w_t}) \cdot \prod_{i=1}^{k} \sigma(-\mathbf{v}'_{w_i} \cdot \mathbf{v}_{w_t})$$	**1. Correct Way to Choose the k Negative Words** For each positive pair (target w_t, context w): • Sample **k words independently** • From the distribution: $$P(w) \propto U(w)^{3/4}$$ Where: • $U(w)$ = unigram frequency of word w • The exponent $\frac{3}{4}$ smooths the distribution This is the method used in the original Word2Vec paper. **2. Why Not Uniform Sampling?** If you sample uniformly: • Rare words get oversampled • Training becomes noisy • Embeddings degrade If you sample directly from raw frequency: • Very common words dominate • Model learns less useful contrasts The $3/4$ power balances this. **3. Important Practical Rules** **Rule 1 — Never Sample the Positive Context Word** Do not allow the true context word to appear as a negative. **Rule 2 — Sample With Replacement** Each of the k negatives is sampled independently. Duplicates are allowed (though uncommon). **Rule 3 — Do Not Manually Pick "Hard" Negatives** Standard Word2Vec does **not** use hard negative mining.	**1. CBOW objective** Given context words $w_1, w_2, \ldots, w_C$, predict the center word w_t. CBOW computes a hidden representation: $$h = \frac{1}{C} \sum_{i=1}^{C} \mathbf{v}_{w_i}$$ • $\mathbf{v}_{w_i}$: embedding of context word **2. Softmax probability (core expression)** The probability of predicting word w is: $$P(w \mid \text{context}) = \frac{\exp(\mathbf{u}_w^\top h)}{\sum_{w' \in V} \exp(\mathbf{u}_{w'}^\top h)}$$ • $\mathbf{u}_w$: output embedding of word w • V: vocabulary
	We usually replce the entire probability computation with negative contrastive learning	

LSTMs and ELMO/Fasttext

Always start with c_0 and h_0 to be vector of all 0s

1 Bidirectional LSTMs in ELMo

ELMo produces **contextual embeddings** by running two LSTMs:

- **Forward LSTM**: reads $w_1 \rightarrow w_T$, modeling

$$p(w_t \mid w_1, \ldots, w_{t-1})$$

- **Backward LSTM**: reads $w_T \rightarrow w_1$, modeling

$$p(w_t \mid w_{t+1}, \ldots, w_T)$$

- Final embedding for word t:

$$\mathrm{ELMo}(t) = [\overrightarrow{h_t}; \overleftarrow{h_t}]$$

with each $\overrightarrow{h_t}, \overleftarrow{h_t} \in \mathbb{R}^H$ → total size $2H$.

2 Loss Function

Total loss = forward LM loss + backward LM loss:

$$\mathcal{L}_{\text{forward}} = -\sum_{t=1}^{T} \log p(w_t \mid w_1, \ldots, w_{t-1})$$

$$\mathcal{L}_{\text{backward}} = -\sum_{t=1}^{T} \log p(w_t \mid w_{t+1}, \ldots, w_T)$$

$$\mathcal{L} = \mathcal{L}_{\text{forward}} + \mathcal{L}_{\text{backward}}$$

3 LSTM Gates & States (with dimensions)

Let:

- E = embedding size (from char-CNN)
- H = LSTM hidden size
- $x_t \in \mathbb{R}^E$ = embedding at step t
- $h_{t-1} \in \mathbb{R}^H$ = previous hidden state
- $c_{t-1} \in \mathbb{R}^H$ = previous cell state

Forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$W_f \in \mathbb{R}^{H \times (H+E)}$, $b_f \in \mathbb{R}^H$, $f_t \in \mathbb{R}^H$

Input gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$W_i \in \mathbb{R}^{H \times (H+E)}$, $b_i \in \mathbb{R}^H$, $i_t \in \mathbb{R}^H$

Candidate cell state:

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$W_c \in \mathbb{R}^{H \times (H+E)}$, $b_c \in \mathbb{R}^H$, $\tilde{c}_t \in \mathbb{R}^H$

Update cell state:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$c_t \in \mathbb{R}^H$

Output gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$W_o \in \mathbb{R}^{H \times (H+E)}$, $b_o \in \mathbb{R}^H$, $o_t \in \mathbb{R}^H$

Hidden state:

$$h_t = o_t \odot \tanh(c_t)$$

$h_t \in \mathbb{R}^H$

4 Computing Probabilities

From h_t (forward or backward):

1. **Project to vocab size V:**

$$\text{logits}_t = W_{\text{vocab}} h_t + b_{\text{vocab}}$$

$W_{\text{vocab}} \in \mathbb{R}^{V \times H}$, $b_{\text{vocab}} \in \mathbb{R}^V$, $\text{logits}_t \in \mathbb{R}^V$

2. **Softmax over vocabulary:**

$$p(w_t = k \mid \text{context}) = \frac{\exp(\text{logits}_t[k])}{\sum_{j=1}^{V} \exp(\text{logits}_t[j])}$$

3. **Loss:**

For the correct word w_t^{true}:

$$\mathcal{L}_t = -\log p(w_t^{\text{true}} \mid \text{context})$$

RNNs

With time series explaination

1 Vanilla RNN Definition

A **Recurrent Neural Network** processes sequences one step at a time, keeping a **hidden state** that carries information from previous steps.

For a single-direction RNN (left → right):

At each time step t:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$

Where:

- $x_t \in \mathbb{R}^E$ = input vector at time t
- $h_{t-1} \in \mathbb{R}^H$ = hidden state from previous step
- $W_h \in \mathbb{R}^{H \times H}$ = recurrent weights
- $W_x \in \mathbb{R}^{H \times E}$ = input weights
- $b_h \in \mathbb{R}^H$ = bias
- $h_t \in \mathbb{R}^H$ = hidden state at time t

No gates, no separate cell state — just **hidden state recurrence.**

2 Probability Computation (Language Modeling Example)

If we want to predict the next token:

1. Project hidden state to vocabulary space:

$$\text{logits}_t = W_{\text{vocab}} h_t + b_{\text{vocab}}$$

$W_{\text{vocab}} \in \mathbb{R}^{V \times H}$, $b_{\text{vocab}} \in \mathbb{R}^V$

2. Apply softmax:

$$p(w_{t+1} = k) = \frac{\exp(\text{logits}_t[k])}{\sum_{j=1}^{V} \exp(\text{logits}_t[j])}$$

3. Cross-entropy loss on true next token.

1 Loss Functions for Time Series with RNNs

Since time series forecasting outputs **continuous values** (regression problem), we don't use NLP's cross-entropy.

Common choices:

Loss Function	Formula	Use Case
Mean Squared Error (MSE)	$\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$	Standard choice, penalizes large errors heavily
Mean Absolute Error (MAE)	(|frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|	Robust to outliers
Huber Loss	Quadratic for small errors, linear for large errors	Good mix of MSE & MAE
RMSE / MAPE / SMAPE	Variants for specific business contexts	Useful for % error evaluation

2 RNNs for Time Series with Features (Multivariate)

Setup

- **Features per step:** E
- **Sequence length:** T
- Input shape for batch: $(\text{batch_size}, T, E)$

Flow

1. At each step t:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$

- $x_t \in \mathbb{R}^E$ → feature vector at time t
- $W_x \in \mathbb{R}^{H \times E}$, $W_h \in \mathbb{R}^{H \times H}$
- $h_t \in \mathbb{R}^H$ → hidden state

2. **Output layer:**
   - For predicting next time step's value(s):

$$\hat{y}_{T+1} = W_o h_T + b_o$$

where:

- $W_o \in \mathbb{R}^{F \times H}$ (F = forecast dimension, often E)
- $b_o \in \mathbb{R}^F$

3. **Loss:**

Compare $\hat{y}_{T+1}$ with y_{T+1} using MSE/MAE/Huber.

3 RNNs for Univariate Time Series

Setup

- Only one **feature** per time step ($E = 1$)
- Sequence length $T \rightarrow (T, 1)$ per example

Flow

1. At each step:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b_h)$$

- $x_t \in \mathbb{R}^1$ (scalar, treated as 1D vector)
- $W_x \in \mathbb{R}^{H \times 1}$, $W_h \in \mathbb{R}^{H \times H}$

2. Output prediction:

$$\hat{y}_{T+1} = W_o h_T + b_o$$

- $W_o \in \mathbb{R}^{1 \times H}$, $b_o \in \mathbb{R}^1$
- Output is scalar

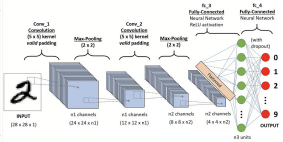
3. **Loss:**

$$\mathcal{L} = \frac{1}{B}\sum_{i=1}^{B}\left(y_{T+1}^{(i)} - \hat{y}_{T+1}^{(i)}\right)^2$$

(MSE example)

CNNs

Start with image (B/W or colour), apply it through filters/kernels whose params will be learned in training which finally gets connected to a full connected network on which softmax is applied to get the final probabilities



1. What a CNN filter really is

A **filter** (or kernel) is just a **small matrix of numbers** (weights) that we slide over the input.

- In images: detects edges, lines, textures
- In LeNet: first filters detect **boundaries** (edges of digits), later filters detect **shapes** (loops, curves)

2. Example — a simple boundary (edge) detecting filter

Let's say our input is a **grayscale image** (pixel values 0–255).

Here's a common **vertical edge filter**:

$$K = \begin{bmatrix} 1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

3. Filters in LeNet

Layer C1 in LeNet:

- Input: 32×32 px grayscale image.
- Filters: each 5×5.
- Learns basic edge detectors like the one above:
  - Some detect vertical boundaries.
  - Some detect horizontal boundaries.
  - Some detect diagonal strokes (important for digits like 4, 7)

Output after C1:

- For each filter, a **28×28 feature map** showing "where in the image this pattern appears strongly".

4. What happens after

- **S2 (Average Pooling):** Reduces size, keeps strongest boundary info.
- **C3:** Second set of filters now works on **the feature maps**, not the raw pixels
  - → Can detect **combinations of edges** (loops, corners, junctions)
- Later layers in LeNet detect **whole digit shapes**.

5. Visual intuition

Imagine you're a detective looking for outlines:

1. First layer says: "There's a vertical edge here, and here, and here."
2. Second layer says: "Those vertical edges plus some horizontal edges form a rectangle."
3. Final layer says: "That rectangle and curve look like the digit '8'."

✅ **Key takeaway:**

Filters are pattern detectors.

In LeNet:

- **Early filters** = edge/boundary detectors
- **Mid filters** = part detectors (loops, corners)
- **Late layers** = object recognizers (whole digits)

Transformers/Attention

This has diagrammatic representation of how attention works

https://claude.ai/chat/b78151f1-5747-4be3-90ec-481009d9fc83

1. Input Representation

We start with a sentence:

```
embone                                                    Copy  Edit
x₁ = "I",  x₂ = "love",  x₃ = "cats"
```

We add a **[CLS] token** at the start (special token for classification):

```
cohorp                                                    Copy  Edit
[CLS], "I", "love", "cats"
```

Each token gets mapped to an **embedding** vector of size d_{model} (say 768):

$$E = \begin{bmatrix} e_{[CLS]} \\ e_I \\ e_{love} \\ e_{cats} \end{bmatrix} \quad \text{where} \quad e_i \in \mathbb{R}^{d_{\text{model}}}$$

2. Creating Q, K, V

For each token embedding e_i, we create:

$$Q_i = e_i W_Q, \quad K_i = e_i W_K, \quad V_i = e_i W_V$$

where:

- $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times d_k}$ are learned weight matrices.
- d_k is usually d_{model}/h (if we have h attention heads).

Stacked for all tokens:

$$Q = EW_Q, \quad K = EW_K, \quad V = EW_V$$

Now:

- $Q \in \mathbb{R}^{n \times d_k}$
- $K \in \mathbb{R}^{n \times d_k}$
- $V \in \mathbb{R}^{n \times d_k}$

where n = number of tokens.

3. Self-Attention

We compute attention scores between every token pair:

$$\text{scores} = \frac{QK^T}{\sqrt{d_k}}$$

- Shape: $(n \times d_k) \cdot (d_k \times n) \rightarrow n \times n$
- Each element scores_{ij} = similarity between token i's **question** and token j's **key**.

4. Attention Weights

Apply softmax along each row to turn scores into probabilities:

$$\alpha_{ij} = \frac{\exp(\text{scores}_{ij})}{\sum_{j'} \exp(\text{scores}_{ij'})}$$

- α_{ij} = how much token i attends to token j.

5. Weighted Sum of Values

Each token output is:

$$z_i = \sum_{j=1}^{n} \alpha_{ij} V_j$$

In matrix form:

$$Z = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Where $Z \in \mathbb{R}^{n \times d_v}$.

6. Multi-Head Attention

Instead of one attention, we have h heads:

$$\text{head}_i = \text{Attention}(QW_Q^{(i)}, KW_K^{(i)}, VW_V^{(i)})$$

Then we concatenate all heads:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

W_O is another learned matrix.

7. Feed Forward + Residual + Norm

For each token:

$$\text{output} = \text{LayerNorm}\,(Z + \text{FeedForward}(Z))$$

- FeedForward is typically:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

8. Stacking Layers

We repeat the attention + feedforward structure for L layers.

Each layer refines token representations.

9. Classification Step

After L layers, take the **[CLS] token vector:**

$$h_{CLS} \in \mathbb{R}^{d_{\text{model}}}$$

Pass it through a classification head:

$$y = \text{softmax}(h_{CLS} W_c + b_c)$$

Positional Encoding

Why Positional Encoding Is Needed

Self-attention treats inputs as a **set of tokens.**

Without position information:

```
"The cat sat"
"The sat cat"
```

would look identical to the model because the same tokens appear.

To solve this, the authors **add a position vector to each token embedding.**

$$Input = TokenEmbedding + PositionalEncoding$$

The Formula Used in the Paper

For a position pos and embedding dimension i:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Where:

- **pos** = token position in the sequence
- **i** = dimension index
- **d_model** = embedding size (e.g., 512)

Interpretation:

- Even dimensions → **sine**
- Odd dimensions → **cosine**
- Different frequencies encode position at multiple scales.

Simple Example (Small Embedding)

Assume:

```
sentence = ["I", "love", "AI"]
embedding size = 4
```

Step 1 — Word embeddings (example values)

Word	Embedding
I	[0.2, 0.4, 0.1, 0.7]
love	[0.6, 0.1, 0.8, 0.2]
AI	[0.3, 0.9, 0.5, 0.4]

Step 2 — Positional encodings

Position	PE vector
0	[0, 1, 0, 1]
1	[0.84, 0.54, 0.01, 0.999]
2	[0.91, -0.42, 0.02, 0.999]

Step 3 — Add them

```
Input to transformer = embedding + positional encoding
```

Word	Final vector
I	[0.2, 1.4, 0.1, 1.7]
love	[1.44, 0.64, 0.81, 1.199]
AI	[1.21, 0.68, 0.82, 1.399]

Now each token vector contains **both meaning + position.**

Yes. In the original Transformer described in Attention Is All You Need, **positional encodings are fixed and depend only on the position index**, not on the sentence or the words.

Key idea

For a given model dimension d_{model}:

- Position 0 always has the same positional vector
- Position 1 always has the same positional vector
- Position 2 always has the same positional vector
- etc.

Therefore, across all sentences:

Position	Positional Encoding (same every time)
0	PE(0)
1	PE(1)
2	PE(2)
3	PE(3)

Those vectors are computed once using the sine/cosine formula and reused.

Example

Two sentences:

```
Sentence A: "I love AI"
Sentence B: "Dogs chase cats"
```

Token positions:

Position	Sentence A	Sentence B	Positional Encoding
0	I	Dogs	PE(0)
1	love	chase	PE(1)
2	AI	cats	PE(2)

Even though the words differ, **the positional encoding vectors are identical** for the same positions.

The model input becomes:

```
token_embedding + positional_encoding
```

Vector search (FAISS, Pinecone etc)	https://claude.ai/chat/7beccfbe-45ec-431f-a113-e7fe1fe9142d	Go through the PDF for complete understanding

- $W_c \in \mathbb{R}^{d_{model} \times C}$ where C = number of classes.
- Output y = probability distribution over classes.

Yes. In the original Transformer described in Attention Is All You Need, **positional encodings are depend only on the position index**, not on the sentence or the words.

Key Idea

For a given model dimension d_{model}:

- Position **0** always has the same positional vector
- Position **1** always has the same positional vector
- Position **2** always has the same positional vector
- etc.

Therefore, across all sentences:

Position	Positional Encoding (same every time)
0	PE(0)
1	PE(1)
2	PE(2)
3	PE(3)

These vectors are computed once using the sine/cosine formula and reused.

Example

Two sentences:

```
Sentence A: "I love AI"
Sentence B: "Dogs chase cats"
```

Token positions:

Position	Sentence A	Sentence B	Positional Encodin
0	I	Dogs	PE(0)
1	love	chase	PE(1)
2	AI	cats	PE(2)

Even though the words differ, **the positional encoding vectors are identical** for the same positi

The model input becomes:

```
token_embedding + positional_encoding
```

Agents vs Workflows

Agent (Claude decides actions)

- An **agent** is a system where Claude **reasons and decides what actions to take**.
- It can **choose tools dynamically** based on the task.
- Works in a loop: **input → reasoning → tool use → result → continue until goal**.
- Used for **complex or open-ended tasks**.

Example flow:

```
Plain text
User question
   ↓
Claude analyzes problem
   ↓
Claude chooses a tool (search docs, query DB)
   ↓
Tool result returned
   ↓
Claude produces final answer
```

Workflow (predefined steps)

- A **workflow** is a **fixed sequence of steps** controlled by the developer.
- Claude is used only for specific tasks (e.g., classification or text generation).
- Execution order is **deterministic**.

Example flow:

```
Plain text
Ticket received
   ↓
Claude classifies issue
   ↓
System retrieves documentation
   ↓
Claude generates response
   ↓
Send reply
```

Practical Rule

Feature	Workflow	Agent
Goal	Yes	Yes
Predefined steps	Yes	No
Tool access	Possible	Required
Model decides next action	No	Yes

RAGs

RAG (Retrieval-Augmented Generation) is an architecture used with large language models (LLMs) where the model retrieves external information and uses it while generating a response. It combines **information retrieval** (searching documents) with **text generation** (LLM output).

Basic Idea

Traditional LLMs rely only on what they learned during training. RAG adds a step that **retrieves relevant documents at query time**.

Process flow

1. **User query**
2. **Retriever searches a knowledge source**
   - vector database
   - document store
   - search engine
3. **Relevant documents are returned**
4. **LLM receives the query + retrieved context**
5. **LLM generates an answer grounded in that context**

```
User Query
   ↓
Retriever (vector search / embeddings)
   ↓
Relevant documents
   ↓
LLM prompt with context
   ↓
Generated answer
```